

C++ Programming

Day 15: Two-Body Gravity Simulator

OOP, Struct, Numerical Integration, Multi-file Project, CSV Output

Contents

1	학습 목표	2
2	프로젝트 개요	2
2.1	프로젝트 파일 구조	2
3	물리 모델	3
4	Particle 클래스	3
4.1	Particle.h	3
4.2	Particle.cpp	4
5	Physics 모듈	4
5.1	물리량 struct	4
5.2	Physics.h	4
5.3	Physics.cpp	5
6	Explicit Euler Method	6
7	Simulation 모듈	6
7.1	Simulation.h	7
7.2	Simulation.cpp	7
8	초기조건과 Simulation Loop	8
9	CSV 출력과 main.cpp	8
9.1	main.cpp	8
10	Build와 실행	9
11	CSV 데이터로 궤도 확인	9
12	프로젝트에서 사용한 C++ 개념	10
13	Numerical Method의 한계	10
14	Day 15 정리	10

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 두 입자의 상태를 `Particle` 클래스로 표현하기
- 위치, 속도, 가속도를 `struct`로 묶어 물리량을 명확하게 표현하기
- 두 입자의 상대위치로 중력가속도를 계산하기
- 두 입자의 운동방정식을 직접 수치적분하기
- Explicit Euler method로 한 timestep을 구현하기
- 헤더와 구현 파일을 분리한 다중 파일 프로젝트 구성하기
- 시뮬레이션 결과를 CSV 파일로 저장하기
- CSV 데이터를 Excel의 XY Scatter chart로 확인하기

2. 프로젝트 개요

Day 15는 C++ 문법을 추가로 학습하는 날이 아니라, 지금까지 배운 내용을 실제 물리 문제에 연결하는 미니프로젝트이다. 프로젝트의 목표는 두 질점 사이에 뉴턴 중력만 작용한다고 가정하고, 두 입자의 운동을 시간에 따라 수치적으로 계산하는 것이다.

프로젝트 목표

두 입자의 질량, 위치, 속도를 초기조건으로 설정하고, 각 timestep에서 중력가속도를 계산한 뒤 Explicit Euler method로 위치와 속도를 갱신한다. 계산된 궤적은 `trajectory.csv`에 저장한다.

2.1. 프로젝트 파일 구조

```
day15/
|-- main.cpp
|-- Particle.h
|-- Particle.cpp
|-- Physics.h
|-- Physics.cpp
|-- Simulation.h
|-- Simulation.cpp
'-- output/
    '-- trajectory.csv
```

핵심 포인트

이 프로젝트에서는 환산질량을 이용해 two-body 문제를 one-body 문제로 환원하지 않는다. 수치 해석에서는 두 입자의 운동방정식을 각각 직접 적분할 수 있으므로, 두 `Particle` 객체의 상태를 그대로 갱신하는 구조를 사용한다.

3. 물리 모델

두 입자의 위치를 각각 $\mathbf{r}_1$, $\mathbf{r}_2$ 라 하고 상대위치를

$$\mathbf{r} = \mathbf{r}_2 - \mathbf{r}_1$$

로 정의한다.

거리 r 은

$$r = \sqrt{x^2 + y^2}$$

이다.

뉴턴의 만유인력 법칙에 따라 두 입자 사이 힘의 크기는

$$F = G \frac{m_1 m_2}{r^2}$$

이고, 벡터형으로 쓰면

$$\mathbf{F}_1 = Gm_1m_2 \frac{\mathbf{r}}{r^3}, \quad \mathbf{F}_2 = -Gm_1m_2 \frac{\mathbf{r}}{r^3}$$

이다.

각 입자의 가속도는

$$\mathbf{a}_1 = Gm_2 \frac{\mathbf{r}}{r^3}, \quad \mathbf{a}_2 = -Gm_1 \frac{\mathbf{r}}{r^3}$$

이 된다.

핵심 포인트

$1/r^2$ 는 힘의 크기이고, 벡터식에서 $\mathbf{r}/r^3$ 가 나타나는 이유는 방향을 나타내는 단위벡터 $\hat{\mathbf{r}} = \mathbf{r}/r$ 를 곱했기 때문이다.

4. Particle 클래스

Particle은 하나의 물체가 가진 질량, 위치, 속도를 저장한다.

4.1. Particle.h

```

1 #ifndef PARTICLE_H
2 #define PARTICLE_H
3
4 class Particle
5 {
6 public:
7     double mass;
8     double x;
9     double y;
10    double vx;
11    double vy;
12
13    Particle(double mass, double x, double y, double vx, double vy);
14
15    void showInfo() const;
16 };
17
18 #endif

```

4.2. Particle.cpp

```

1 #include <iostream>
2 #include "Particle.h"
3
4 using namespace std;
5
6 Particle::Particle(double mass, double x, double y, double vx, double vy)
7 {
8     this->mass = mass;
9     this->x = x;
10    this->y = y;
11    this->vx = vx;
12    this->vy = vy;
13 }
14
15 void Particle::showInfo() const
16 {
17     cout << "Mass: " << mass << endl;
18     cout << "Position: (" << x << ", " << y << ")" << endl;
19     cout << "Velocity: (" << vx << ", " << vy << ")" << endl;
20 }

```

5. Physics 모듈

물리 계산은 Particle 클래스와 분리하여 Physics.h, Physics.cpp에서 처리한다.

5.1. 물리량 struct

위치, 속도, 가속도는 서로 다른 의미를 가진 물리량이므로 별도의 구조체로 분리한다.

```

1 struct Position
2 {
3     double x;
4     double y;
5 };
6
7 struct Velocity
8 {
9     double vx;
10    double vy;
11 };
12
13 struct Acceleration
14 {
15     double ax;
16     double ay;
17 };
18
19 struct Accelerations
20 {
21     Acceleration acceleration1;
22     Acceleration acceleration2;
23 };

```

5.2. Physics.h

```
1 #ifndef PHYSICS_H
2 #define PHYSICS_H
3
4 #include "Particle.h"
5
6 struct Position
7 {
8     double x;
9     double y;
10 };
11
12 struct Velocity
13 {
14     double vx;
15     double vy;
16 };
17
18 struct Acceleration
19 {
20     double ax;
21     double ay;
22 };
23
24 struct Accelerations
25 {
26     Acceleration acceleration1;
27     Acceleration acceleration2;
28 };
29
30 Position getRelativePosition(
31     const Particle& particle1,
32     const Particle& particle2
33 );
34
35 Accelerations getAccelerations(
36     const Particle& particle1,
37     const Particle& particle2
38 );
39
40 #endif
```

5.3. Physics.cpp

```
1 #include "Physics.h"
2 #include <cmath>
3
4 Position getRelativePosition(
5     const Particle& particle1,
6     const Particle& particle2
7 )
8 {
9     Position relativePosition;
10    relativePosition.x = particle2.x - particle1.x;
11    relativePosition.y = particle2.y - particle1.y;
12
13    return relativePosition;
14 }
15
```

```

16 Accelerations getAccelerations(
17     const Particle& particle1,
18     const Particle& particle2
19 )
20 {
21     Position relativePosition = getRelativePosition(particle1, particle2);
22
23     double x = relativePosition.x;
24     double y = relativePosition.y;
25     double m1 = particle1.mass;
26     double m2 = particle2.mass;
27
28     double r = std::sqrt(x * x + y * y);
29
30     const double G = 6.67430e-11;
31
32     Accelerations accelerations;
33
34     accelerations.acceleration1.ax = G * m2 * x / (r * r * r);
35     accelerations.acceleration1.ay = G * m2 * y / (r * r * r);
36
37     accelerations.acceleration2.ax = -G * m1 * x / (r * r * r);
38     accelerations.acceleration2.ay = -G * m1 * y / (r * r * r);
39
40     return accelerations;
41 }

```

핵심 포인트

두 입자의 위치가 완전히 같아지면 $r = 0$ 이 되어 분모가 0이 된다. 현재 프로젝트에서는 초기조건을 서로 다른 위치로 설정하고 충돌 처리는 구현하지 않는다.

6. Explicit Euler Method

현재 상태에서 가속도를 계산한 뒤 timestep Δt 만큼 전진한다.

위치는

$$x_{n+1} = x_n + v_{x,n}\Delta t, \quad y_{n+1} = y_n + v_{y,n}\Delta t$$

로 갱신한다.

속도는

$$v_{x,n+1} = v_{x,n} + a_{x,n}\Delta t, \quad v_{y,n+1} = v_{y,n} + a_{y,n}\Delta t$$

로 갱신한다.

핵심 포인트

Explicit Euler에서는 새 위치와 새 속도를 모두 기존 상태 n 을 이용해 계산해야 한다. 속도를 먼저 갱신한 뒤 그 새 속도로 위치를 갱신하면 Semi-Implicit Euler가 된다.

7. Simulation 모듈

step()은 두 입자의 상태를 reference로 받아 한 timestep만큼 직접 변경한다.

7.1. Simulation.h

```
1 #ifndef SIMULATION_H
2 #define SIMULATION_H
3
4 #include "Particle.h"
5 #include "Physics.h"
6
7 void step(Particle& particle1, Particle& particle2, double dt);
8
9 #endif
```

7.2. Simulation.cpp

```
1 #include "Simulation.h"
2
3 void step(Particle& particle1, Particle& particle2, double dt)
4 {
5     Accelerations accelerations
6         = getAccelerations(particle1, particle2);
7
8     Position oldPosition1;
9     oldPosition1.x = particle1.x;
10    oldPosition1.y = particle1.y;
11
12    Position oldPosition2;
13    oldPosition2.x = particle2.x;
14    oldPosition2.y = particle2.y;
15
16    Velocity oldVelocity1;
17    oldVelocity1.vx = particle1.vx;
18    oldVelocity1.vy = particle1.vy;
19
20    Velocity oldVelocity2;
21    oldVelocity2.vx = particle2.vx;
22    oldVelocity2.vy = particle2.vy;
23
24    Position newPosition1;
25    newPosition1.x = oldPosition1.x + oldVelocity1.vx * dt;
26    newPosition1.y = oldPosition1.y + oldVelocity1.vy * dt;
27
28    Position newPosition2;
29    newPosition2.x = oldPosition2.x + oldVelocity2.vx * dt;
30    newPosition2.y = oldPosition2.y + oldVelocity2.vy * dt;
31
32    Velocity newVelocity1;
33    newVelocity1.vx = oldVelocity1.vx
34                    + accelerations.acceleration1.ax * dt;
35    newVelocity1.vy = oldVelocity1.vy
36                    + accelerations.acceleration1.ay * dt;
37
38    Velocity newVelocity2;
39    newVelocity2.vx = oldVelocity2.vx
40                    + accelerations.acceleration2.ax * dt;
41    newVelocity2.vy = oldVelocity2.vy
42                    + accelerations.acceleration2.ay * dt;
43
44    particle1.x = newPosition1.x;
```



```

45 particle1.y = newPosition1.y;
46 particle1.vx = newVelocity1.vx;
47 particle1.vy = newVelocity1.vy;
48
49 particle2.x = newPosition2.x;
50 particle2.y = newPosition2.y;
51 particle2.vx = newVelocity2.vx;
52 particle2.vy = newVelocity2.vy;
53 }

```

8. 초기조건과 Simulation Loop

테스트에서는 태양과 지구에 가까운 질량, 거리, 속도를 사용한다.

항목	Particle 1	Particle 2
질량	1.989×10^{30} kg	5.972×10^{24} kg
초기 위치	(0,0) m	$(1.496 \times 10^{11}, 0)$ m
초기 속도	(0,0) m/s	(0, 29780) m/s

시간 간격은

$$\Delta t = 3600 \text{ s}$$

로 두어 한 번의 `step()`이 1시간을 의미하도록 한다.

9. CSV 출력과 main.cpp

CSV 파일의 첫 행은 각 열의 의미를 나타낸다.

time,x1,y1,x2,y2

9.1. main.cpp

```

1  #include <iostream>
2  #include <fstream>
3  #include "Particle.h"
4  #include "Physics.h"
5  #include "Simulation.h"
6
7  using namespace std;
8
9  int main()
10 {
11     Particle particle1(
12         1.989e30,
13         0.0, 0.0,
14         0.0, 0.0
15     );
16
17     Particle particle2(
18         5.972e24,
19         1.496e11, 0.0,
20         0.0, 29780.0
21     );
22
23     double dt = 3600.0;

```

```

24 double totalTime = 365.25 * 24.0 * 3600.0;
25
26 int steps = static_cast<int>(totalTime / dt);
27
28 ofstream file("output/trajectory.csv");
29
30 if (!file.is_open())
31 {
32     cout << "Failed to open file" << endl;
33     return 1;
34 }
35
36 file << "time,x1,y1,x2,y2" << endl;
37
38 for (int i = 0; i <= steps; i++)
39 {
40     double time = i * dt;
41
42     file << time << ","
43         << particle1.x << ","
44         << particle1.y << ","
45         << particle2.x << ","
46         << particle2.y << endl;
47
48     if (i < steps)
49     {
50         step(particle1, particle2, dt);
51     }
52 }
53
54 file.close();
55
56 cout << "Simulation completed." << endl;
57 cout << "Data saved to output/trajectory.csv" << endl;
58
59 return 0;
60 }

```

10. Build와 실행

모든 구현 파일을 함께 컴파일해야 한다.

```
1 g++ main.cpp Particle.cpp Physics.cpp Simulation.cpp -o simulation
```

실행 후 output/trajectory.csv가 생성되면 시뮬레이션이 정상적으로 완료된 것이다.

핵심 포인트

main.cpp만 컴파일하면 Particle.cpp, Physics.cpp, Simulation.cpp에 있는 구현을 linker가 찾지 못해 undefined reference 오류가 발생한다.

11. CSV 데이터로 궤도 확인

CSV 자체는 그래프를 저장하는 형식이 아니므로 Excel에서 데이터를 불러온 뒤 XY Scatter chart를 사용한다.

Particle 2의 궤도는

- X 값: x_2
- Y 값: y_2

로 설정한다.

핵심 포인트

Excel이 과학적 표기법 데이터를 문자열로 인식하면 셀 앞에 작은따옴표가 붙을 수 있다. 이 경우 숫자로 변환한 뒤 XY Scatter chart를 생성해야 한다. 단순 Line chart를 사용하면 행 번호나 시간에 대한 그래프가 되어 실제 궤도가 아니다.

12. 프로젝트에서 사용한 C++ 개념

개념	프로젝트에서의 역할
Class	Particle 객체 정의
Struct	Position, Velocity, Acceleration 데이터 묶음
Reference	step()에서 원본 Particle 상태 변경
Const reference	물리 계산 함수에서 불필요한 복사 방지
Header / cpp 분리	선언과 구현 분리
Function	물리 계산과 시뮬레이션 역할 분리
File I/O	CSV 출력
Loop	timestep 반복
static_cast	반복 횟수를 정수로 변환

13. Numerical Method의 한계

Explicit Euler method는 구현이 단순하지만 장시간 궤도 계산에서 에너지 오차가 누적될 수 있다. 따라서 원에 가까운 초기조건을 사용해도 궤도가 완전히 닫히지 않거나 점점 변형될 수 있다.

향후 확장에서는 다음 방법을 비교할 수 있다.

- timestep Δt 감소
- Semi-Implicit Euler
- Velocity Verlet
- Runge-Kutta method
- total energy와 angular momentum의 시간 변화 확인

14. Day 15 정리

Day 15에서는 두 입자 중력 문제를 C++ 다중 파일 프로젝트로 구현했다. 핵심은 물리식을 코드로 직접 변환하고, 각 timestep에서 두 입자의 가속도, 속도, 위치를 순서대로 계산하는 것이다.

프로그램의 전체 흐름은 다음과 같다.

```
Initial Conditions
|
v
Particle 1 + Particle 2
|
v
```

```
Relative Position
|
v
Gravitational Accelerations
|
v
Explicit Euler step
|
v
Updated Particle States
|
v
Repeat
|
v
CSV output
```

이 프로젝트를 통해 클래스, 구조체, 참조, 함수 분리, 헤더 파일, 수치해석, 파일 입출력을 하나의 물리 시뮬레이션 프로그램 안에서 연결했다.